i) N-Queens (Branch and Bound)

```
def solveNQueens(n):
    col = [0]*n
    diag1 = [0]*(2*n)
    diag2 = [0]*(2*n)
    board = [["."]*n for _ in range(n)]
    res = []
    def backtrack(r):
        if r == n:
            res.append(["".join(row) for row in board])
            return
        for c in range(n):
            if col[c] or diag1[r+c] or diag2[r-c+n]:
                continue
            board[r][c] = "Q"
            col[c] = diag1[r+c] = diag2[r-c+n] = 1
            backtrack(r+1)
            board[r][c] = "."
            col[c] = diag1[r+c] = diag2[r-c+n] = 0
    backtrack(0)
    return res
print(solveNQueens(4))
```

ii) Graph Coloring (Backtracking)

```
def graphColoring(graph, m):
    n = len(graph)
    colors = [0] * n
    def isSafe(v, c):
        for i in range(n):
            if graph[v][i] and colors[i] == c:
                return False
        return True

    def solve(v):
        if v == n: return True
        for c in range(1, m+1):
            if isSafe(v, c):
                colors[v] = c
                if solve(v+1): return True
                colors[v] = 0
        return False
    return solve(0), colors

graph = [[0,1,1,1],[1,0,1,0],[1,1,0,1],[1,0,1,0]]
print(graphColoring(graph, 3))
```

iii) Sudoku (Backtracking)

```
def solveSudoku(board):
    def isValid(r, c, ch):
        for i in range(9):
            if board[i][c] == ch or board[r][i] == ch:
                return False
            if board[3*(r//3)+i//3][3*(c//3)+i%3] == ch:
                return False
        return True
    def solve():
        for r in range(9):
            for c in range(9):
                if board[r][c] == ".":
                    for ch in "123456789":
                        if isValid(r, c, ch):
                            board[r][c] = ch
                            if solve(): return True
                            board[r][c] = "."
                    return False
        return True
    solve()
    return board
```

i) N-Queens (for n=4)

.Q.. ..Q.
...Q Q...
Q... ...Q
..Q. .Q..

ii) Graph Coloring (with 3 colors)

Vertex 0 → Color 1
Vertex 1 → Color 2
Vertex 2 → Color 3
Vertex 3 → Color 2

iii) Sudoku Solution (Solved 9×9 Grid)

5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7

8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6

9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9